

slam-toolbox Mapping Algorithm

slam-toolbox Mapping Algorithm

1. Course Content
2. slam-toolbox Introduction
 - 2.1 Core Technical Principles
 - 2.1.1 Front-end Processing (Laser Scan Matching)
 - 2.1.2 Back-end Optimization (Graph Optimization)
3. Running the Demo
 - 3.1 Mapping Process
 - 3.2 Saving the Occupancy Grid Map
 - 3.3 Saving a Pose Graph in posegraph Format
6. Configuration Parameters

1. Course Content

Basics: run the demo program and use the slam-toolbox mapping algorithm to build and save an occupancy grid map.

Advanced: understand the configuration parameters of slam-toolbox

2. slam-toolbox Introduction

slam-toolbox is an open-source ROS-based 2D SLAM (Simultaneous Localization and Mapping) algorithm package, mainly used for mapping and localization of mobile robots in unknown environments. It is based on the **Graph Optimization** framework and combines sensor data such as LiDAR and IMU to build a 2D occupancy grid map of the environment and update the robot pose in real time.

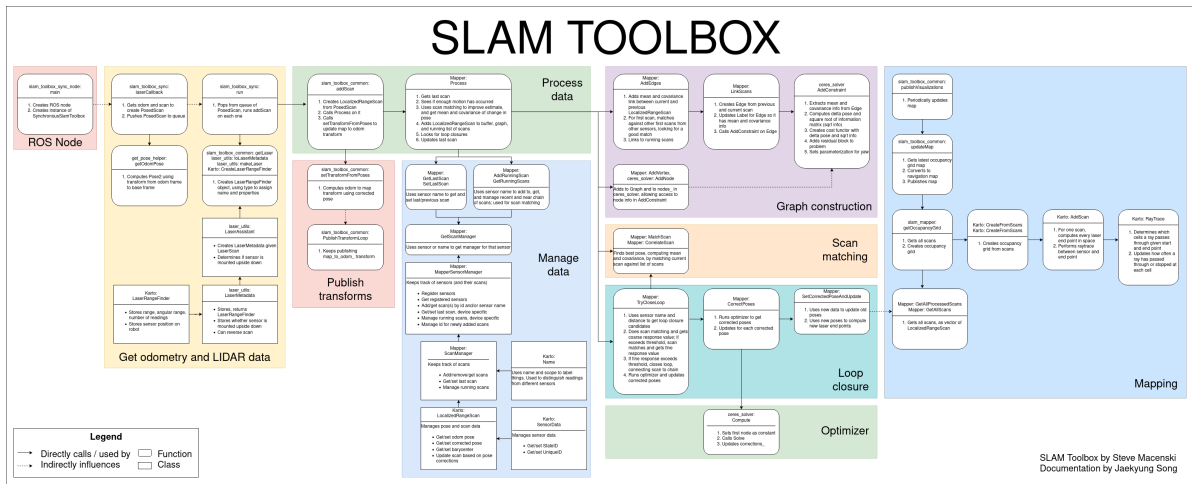
2.1 Core Technical Principles

2.1.1 Front-end Processing (Laser Scan Matching)

- **GICP algorithm:** it estimates the relative pose change of the robot by iteratively computing the optimal match between the current laser scan and the map. Compared with traditional ICP, GICP introduces a probabilistic model and is more robust to outliers.
- **Motion compensation:** uses IMU data to compensate for laser scan distortion during robot motion, improving matching accuracy in dynamic environments.

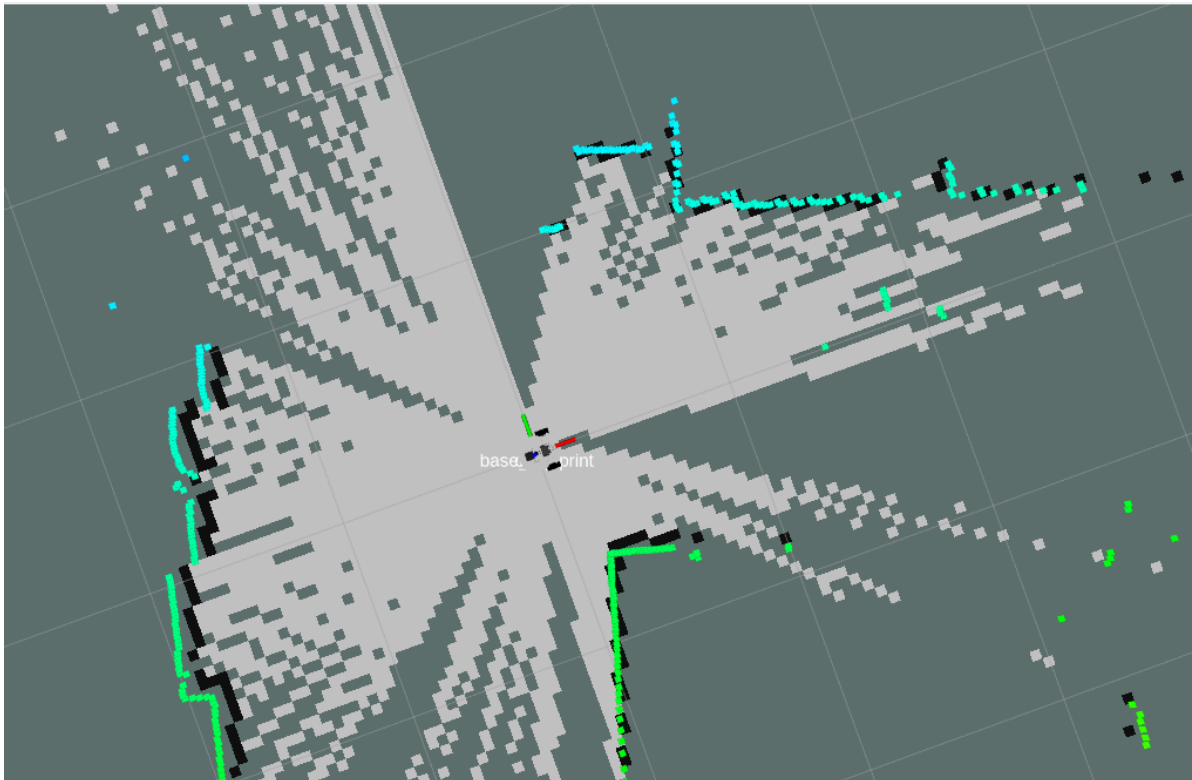
2.1.2 Back-end Optimization (Graph Optimization)

- **Pose graph construction:** the robot pose nodes and the relative pose constraints obtained from scan matching form a graph structure.
- **Global optimization:** uses a nonlinear optimization algorithm (such as Ceres Solver) to optimize the pose graph, eliminating accumulated errors and achieving global consistency of the map.



3. Running the Demo

```
ros2 launch microros_v2_bringup slam_toolbox.launch.py
```

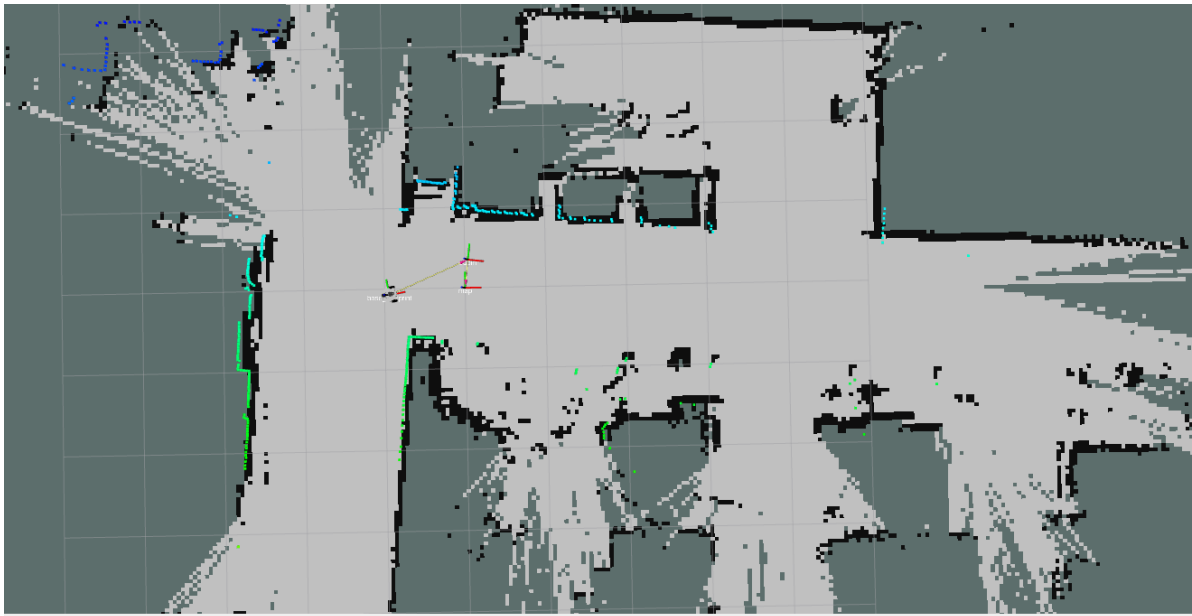


3.1 Mapping Process

- Control the robot to move. You can use keyboard control or gamepad control; keyboard control is used here as an example.

```
ros2 run common_demo keyboard_control
```

- Press I, <, J, L to control the robot to move forward, backward, turn left, and turn right respectively; move the robot slowly to complete mapping



3.2 Saving the Occupancy Grid Map

- Run the following command in the terminal to save the map

```
ros2 launch microros_v2_bringup save_map.launch.py
```

[!TIP]

Optional parameters:

`map_name`: specify the name of the map to save; the default value is map

For example, to save a map named test.yaml, add the parameter: `ros2 launch microros_v2_bringup save_map.launch.py map_name:=test`

A **Map saved successfully** message in the terminal means the map was saved successfully.

3.3 Saving a Pose Graph in posegraph Format

- The posegraph pose graph can be used for route network navigation in later chapters and for navigation using the slam_toolbox localization mode

```
ros2 launch microros_v2_bringup save_map.launch.py map_type:=posegraph
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup save_map.launch.py map_type:=posegraph
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-27-20-41-51-140658-yahboom-201996
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [ros2-1]: process started with pid [201997]
[ros2-1] waiting for service to become available...
[ros2-1] requester: making request: slam_toolbox.srv.SerializePoseGraph_Request(filename='/home/yahboom/map/map')
[ros2-1] response:
[ros2-1] slam_toolbox.srv.SerializePoseGraph_Response(result=0)
[ros2-1]
[INFO] [ros2-1]: process has finished cleanly [pid 201997]
```

6. Configuration Parameters

- Parameter file path for synchronous mapping mode:

```
$HOME/microros_ws/src/microros_v2_bringup/config/mapper_params_online_async.yaml
```

- See the comments for the meaning of each parameter

```
sync_slam_toolbox:
  ros__parameters:

    # Plugin parameters (optimization solver_plugin:
    solver_plugins::CeresSolver # Optimization solver plugin, uses Ceres solver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY # Ceres linear solver type:
    sparse normal Cholesky decomposition
    ceres_preconditioner: SCHUR_JACOBI # Ceres preconditioner type: Schur-
    Jacobi preconditioning
    ceres_trust_strategy: LEVENBERG_MARQUARDT # Ceres trust region strategy:
    Levenberg-Marquardt method
    ceres_dogleg_type: TRADITIONAL_DOGLEG # Ceres dogleg type: traditional
    dogleg (only effective when the trust region strategy is DOGLEG)
    ceres_loss_function: None # Ceres loss function: none (uses the default
    squared loss)

    # ROS parameters
    odom_frame: odom # Odometry frame name
    map_frame: map # Map frame name
    base_frame: base_footprint # Robot base frame name
    scan_topic: /scan # LiDAR scan data topic (absolute path)
    use_map_saver: true # whether to enable the map saver service and subscribe
    to the map topic
    mode: mapping # Operating mode: mapping or localization

    # Map loading related parameters (if enabled, loads a map from the specified
    file and continues mapping)
    map_file_name: "" # Pose graph file name to load (empty means no loading)
    # map_start_pose: [0.0, 0.0, 0.0] # Starting pose [x, y, yaw] when loading
    the map (mutually exclusive with map_start_at_dock; this parameter takes
    priority)
    map_start_at_dock: true # whether to start loading from the first node of
    the map (dock position)

    debug_logging: false # whether to enable debug log output
    throttle_scans: 1 # Scan data throttle count in synchronous mode (process 1
    of every n frames)
    transform_publish_period: 0.02 # Publish period of the map-to-odometry
    transform (s); 0 means no publishing
    map_update_interval: 5.0 # Update interval of the 2D occupancy grid map (s,
    for visualization or other applications)
    resolution: 0.05 # Resolution of the 2D occupancy grid map (m/pixel)
    min_laser_range: 0.05 # Minimum valid laser distance used for grid map
    generation (m)
    max_laser_range: 12.0 # Maximum valid laser distance used for grid map
    generation (m)
```

```

    minimum_time_interval: 0.5 # Minimum time interval for processing two scan
frames in synchronous mode (s)
    transform_timeout: 0.2 # Timeout for TF transform lookups (s)
    tf_buffer_duration: 30. # TF message cache duration (s)
    stack_size_to_use: 40000000 # Program stack size (bytes), used for
serialization/deserialization of large maps
    enable_interactive_mode: true # Whether to enable interactive mode (keeps
the laser scan cache for RViz interaction; increases memory usage)

# General parameters
    use_scan_matching: true # Whether to use scan matching
    use_scan_barycenter: true # Whether to use the scan barycenter (improves
matching stability)
    minimum_travel_distance: 0.5 # Minimum distance the robot must travel
before processing new scan data (m)
    minimum_travel_heading: 0.5 # Minimum angle the robot must rotate before
processing new scan data (radians)
    scan_buffer_size: 10 # Scan data buffer size (keeps the most recent n scan
frames)
    scan_buffer_maximum_scan_distance: 10.0 # Maximum distance of laser points
in the scan buffer (m, points beyond this are filtered)
    link_match_minimum_response_fine: 0.1 # Minimum response of link matching
in fine matching (used for constraints between adjacent nodes)
    link_scan_maximum_distance: 1.5 # Maximum distance between adjacent scans
(m, used to build links between nodes)
    loop_search_maximum_distance: 3.0 # Maximum search distance for loop
closure detection (m)
    do_loop_closing: true # Whether to enable loop closure detection
    loop_match_minimum_chain_size: 10 # Minimum node chain length required for
loop closure detection (avoids false matches from short chains)
    loop_match_maximum_variance_coarse: 3.0 # Maximum variance threshold during
coarse matching (used to filter candidate loop closures)
    loop_match_minimum_response_coarse: 0.35 # Minimum response of loop
closures in coarse matching
    loop_match_minimum_response_fine: 0.45 # Minimum response of loop closures
in fine matching

# Correlation parameters - scan matching parameters
    correlation_search_space_dimension: 0.5 # Search space size for scan
matching (m, searches for the best match within this range)
    correlation_search_space_resolution: 0.01 # Resolution of the scan matching
search space (m, affects matching accuracy and speed)
    correlation_search_space_smear_deviation: 0.1 # Smear deviation of laser
points in scan matching (m, increases robustness)

# Correlation parameters - loop closure detection parameters
    loop_search_space_dimension: 8.0 # Search space size for loop closure
detection (m)
    loop_search_space_resolution: 0.05 # Resolution of the loop closure search
space (m)
    loop_search_space_smear_deviation: 0.03 # Smear deviation of laser points
in loop closure detection (m)

# Scan matcher parameters
    distance_variance_penalty: 0.5 # Distance variance penalty coefficient
(affects the weight of distance errors in matching)
    angle_variance_penalty: 1.0 # Angle variance penalty coefficient (affects
the weight of angle errors in matching)

```

```
    fine_search_angle_offset: 0.00349 # Angle offset for fine search (radians,
about 0.2 deg)
    coarse_search_angle_offset: 0.349 # Angle offset for coarse search
(radians, about 20 deg)
    coarse_angle_resolution: 0.0349 # Angular resolution for coarse search
(radians, about 2 deg)
    minimum_angle_penalty: 0.9 # Minimum angle penalty coefficient (filters
matches with excessive angle deviation)
    minimum_distance_penalty: 0.5 # Minimum distance penalty coefficient
(filters matches with excessive distance deviation)
    use_response_expansion: true # whether to use response expansion (improves
matching stability)
    min_pass_through: 2 # Minimum number of points passing through in scan
matching (filters invalid matches)
    occupancy_threshold: 0.1 # Occupancy grid threshold (used to determine
whether a cell is occupied)
```